

Indivisa

Corporate actions, settled in one atomic batch, without exposing the register.

Settled on Canton DevNet · run it yourself in one command

A distribution is N separate payments.

There is no moment at which the run is definitively finished. Some holders are paid, some are not, and the difference is worked by hand for days afterwards.

\$3.42m

a year in asset-servicing errors, per market participant

3–10%

error rates, described as routine

<40%

straight-through processing on voluntary events

+23%

annual growth in asset-servicing costs

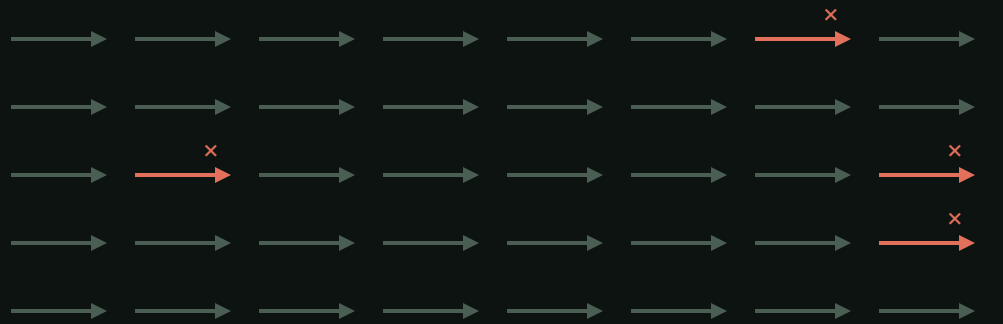
And it cannot simply move onto a public chain.

Paying a coupon publicly publishes the register. Every holder's position size becomes readable by every other holder, by their competitors, and by anyone else, at the moment the coupon pays.

- A bondholder register is **not public information**, and in many markets is protected by data-protection or banking-secrecy rules.
- It is commercially sensitive in all of them: it is the client list.
- That is why this workflow has not moved to a transparent ledger.

One transaction. Every holder paid at the same instant, or nobody.

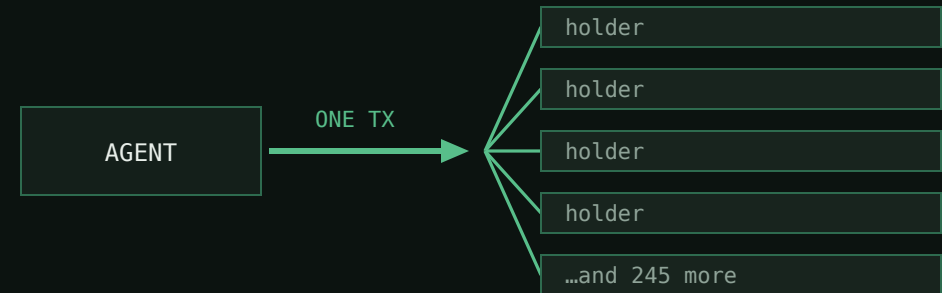
TODAY



250 PAYMENTS · SOME COME BACK

Chased by hand. Claims for weeks. Never definitively done.

INDIVISA



ALL PAID, OR NONE

Nothing partial. Nothing to reconcile. No holder sees another.

THE VALUE, IN THE OPERATIONS MANAGER'S TERMS

The run is finished when the transaction commits.

The event closes the same day

Not after working the **29% of income events that do not go straight through**, on Canton's own published figures. The transaction commits, and that is the answer.

The exception queue is empty

There is no partial state to work. The class of exception "some paid, some not" stops existing.

Nothing to chase next week

No claims tail, no reconciliation against four systems, no explaining to the issuer what happened.

And the register never leaves the paying agent.

Four steps, and only the last one moves money.

- 01 **Once, at onboarding:** each holder signs one standing agreement with the paying agent.
- 02 **Record date:** the register is snapshotted and entitlements calculated, rounded so the parts sum to the announced total exactly.
- 03 **Before payment:** the agent sets aside the cash and prepares one allocation per holder.
- 04 **Payment date:** one settlement. Every leg, or none.

Token Standard V2 made this possible in June.

CIP-0112, approved 12 June 2026: multi-leg settlement where the executor sees every leg and each participant sees only its own. Its worked example is trading and netting. **CIP-112 did not design this. It enabled it.**

HOLDER A	HOLDER B	PAYING AGENT · EXECUTOR
my coupon1,840.00	nothing else here	holder A · 1,840.00
nothing else here	my coupon46,320.00	holder B · 46,320.00
nothing else here	nothing else here	...and 248 more, every one

The striped rows are not hidden. Those amounts were never delivered to that node.

CANTON DEVNET · 24 SEPTEMBER 2026

It ran on the real network.

Five holders paid in a single transaction, 8,421.88 USD. The deliberate failure was run first and refused: one holder not ready, nobody paid.

```
1220652e2e4d32822c39d2ad72088e163eed04718ad1108998001b01aa3483ff466b
```

That is a real update id on the public test network. Full record: [docs/devnet-run.md](#)

Don't take our word for it. Run it.

```
cd judge && docker compose up
```

- A five-participant Canton network, the real contracts, twenty holders.
- One holder deliberately left out. Press the button: refused, and the ledger names them.
- Fix it, press again: settled.
- Nothing to install but Docker. No account anywhere.

0
OF 250 PAID

250 requested
0 executed

Not 249.

Measured, not estimated.

HOLDERS IN ONE TRANSACTION	SETTLES IN	TRANSACTION SIZE
250	1.6 s	440 KB
1,000	11.1 s	1.67 MB

A thousand holders: **days of reconciliation become 11.1 seconds**, and it is final.

In institutional systems, four-eyes authorisation is a requirement, not a feature.

A run can name an approver: a decentralised party, through BitSafe's Decentralization Manager. One optional field. The product is unchanged without it.

One approval → the ledger refuses

Two of three → settles

The threshold is enforced by the ledger, not by our code. In the local demo all three nodes are ours; on DevNet the second is BitSafe's. Four-eyes needs a way back out: a request can be withdrawn until it executes, and both the asking and the withdrawal stay in the ledger history. After settlement there is nothing to withdraw – the correction is a new payment, not an edit.

What we are not claiming.

REAL

The ledger, the contracts, the settlement, the refusal, the privacy, the DevNet update id, the governance engine and its threshold. Every number on screen.

SIMULATED

The cash is TestTokenV2, the standard's own reference asset, not Canton Coin, chosen because it is signed by owner *and* admin, so it forces the same receiver-authorisation case. Holders and positions are generated. Three approver nodes on one machine are not three independent operators.

NOT ATTEMPTED

Corporate-action announcement data. Chainlink, with DTCC and Swift, is attacking that layer. We do the payment layer, and saying so makes the position sharper.

KNOWN, DESIGNED, NOT YET BUILT

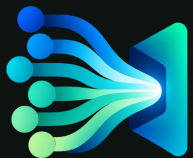
Scoping a run is the first thing we build next.

If 5 of 250 holders are not ready, the batch refuses for all 250. A paying agent needs to settle the 245 today and carry the 5 into a second run. **The batch stays all-or-nothing; its membership becomes a decision, not an accident.** The model supports it. The workflow, and the audit trail that says why five were held back, are not written.

- **Access control:** an operator signs in before the console reads or writes anything, and the check sits in the proxy that holds the credentials, not in the page. There are no roles yet — no maker-checker, where one person prepares a run and another releases it. For a payout that is the control an auditor asks about first.
- **The set-up surface:** the console settles a run; it does not keep the register. Holders, positions and the schedule come from systems a paying agent already runs, and in the demo a script stands in for them. That boundary is deliberate — it is why Indivisa works against registers that already exist.
- **Production onboarding:** holders sign through a real client, not a script.
- **The scheduled agent:** today a person presses the button; in production a daemon watches payment dates.
- **Real cash:** a production run settles Canton Coin or a registry's own token. The model never names the asset, so this is a field and one registry adapter, not a rewrite.
- **MainNet:** the DevNet run proves the path; MainNet is a deployment decision, not a code change. We run validator infrastructure, and it is the first thing after submission.

The buyer is the independent debt paying agent and the corporate trustee, first.

- A few hundred firms worldwide: **independent debt paying agents, corporate trustees, and the issuer-services arms of custodian banks.** Small, named, and each running thousands of events a year.
- **Dividends and redemptions are the same engine:** the same run, a different event type.
- **It works against registers that already exist.** The bond is not tokenized; only the cash is a Token Standard V2 asset.
- **The benchmark is a contribution in its own right:** the first published figures for CIP-112 batch settlement, going to the Canton forum whatever happens here.



Indivisa

One transaction. Every holder paid at the same instant, or nobody. And no holder sees another.

Settled on Canton DevNet · 1220652e...f466b
Run it yourself: `cd judge && docker compose up`